Лекция 9. Реестр моделей и управляемый выпуск

Курс: «Управление процессами машинного обучения MLOps» Продолжительность: 2 академических часа Связанная работа: ЛР10 «MLflow Registry, release manifest и rollback» Актуализация: 20 октября 2026 года

Цель лекции

Показать, как превратить удачный эксперимент в управляемый объект выпуска. После занятия студент должен различать run, logged model, registered model и model version; применять aliases и tags вместо устаревших stages; связывать версию модели с данными, кодом, проверками и решением человека; проектировать promotion и rollback без переобучения модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От run к выпуску	Разобрать риск «лучшая метрика означает production»
10–24 мин	Объекты MLflow Registry	Сопоставить run, logged model, registered model и version
24–38 мин	Signature, environment и происхождение	Показать минимальный проверяемый комплект версии
38–52 мин	Aliases и tags	Переназначить `candidate` и `champion`
52–65 мин	Quality gate и approval	Отделить автоматическую проверку от решения о выпуске
65–76 мин	Release manifest	Проследить commit, data hash, run и model version
76–84 мин	Rollback	Вернуть alias на предыдущую версию без retraining
84–90 мин	Связь с ЛР10	Назначить индивидуальные сценарии и evidence

1. Почему артефакта модели недостаточно

Файл `model.joblib` отвечает только на вопрос «какие байты сохранены». Он не сообщает, на каких данных и каким кодом выполнено обучение, какие параметры использованы, какой контракт входа ожидается, какие проверки пройдены и кто разрешил выпуск. Копирование файла под именем `final_v7_really_final.joblib` не создаёт процесс управления версиями.

Начните с инцидента: сервис загрузил новый файл, метрика в offline-отчёте выросла, но входной schema изменился. Без signature и связи с run команда не может быстро доказать совместимость. Без предыдущей версии и атомарного указателя rollback превращается в поиск резервной копии.

2. Четыре разных объекта

Объект	Что фиксирует	Типичный идентификатор
Experiment run	один запуск, параметры, метрики и артефакты	`run_id`
Logged model	модель внутри run с flavor, signature и environment	`runs:/<run_id>/model`
Registered model	устойчивое логическое имя продукта	`flight-delay-classifier`
Model version	неизменяемая запись о конкретной модели	имя + номер версии

Один run может содержать несколько артефактов. Один registered model может иметь много версий. Номер версии не означает качество: версия 12 может быть хуже версии 8. Поэтому выбор обслуживаемой версии выражается policy и alias, а не максимальным номером.

3. Tracking и Registry решают разные задачи

Tracking помогает сравнивать эксперименты. Registry управляет идентичностью и жизненным циклом выпуска. Переход между ними — отдельное решение: модель должна пройти проверки, получить описание происхождения и быть зарегистрирована. Не регистрируйте каждый пробный run автоматически: реестр станет ещё одним неструктурированным каталогом.

Для локальной учебной работы MLflow Registry требует database-backed backend store. Используйте SQLite URI и один локальный server или прямой клиент. Каталог артефактов и база метаданных являются разными частями; резервная копия только одного из них неполна.

4. Model signature как исполнимый контракт

Signature описывает ожидаемые входы и выходы модели. Она не заменяет предметный data contract, потому что обычно не выражает момент доступности признака, источник, допустимое запаздывание или бизнес-смысл. Но signature позволяет обнаружить несовместимые имена и типы на границе inference.

Покажите цепочку:

1. взять репрезентативный `input_example` без персональных данных;
2. получить прогноз тем же объектом модели;
3. вызвать `infer_signature(inputs, outputs)`;
4. сохранить signature вместе с logged model;
5. на загрузке выполнить smoke prediction.

Отсутствующая signature — стоп-фактор выпуска в ЛР10.

5. Environment и воспроизводимость

MLflow model содержит сведения о среде, но они не отменяют lock-файл и контейнер. Версия Python, библиотек, системные зависимости и CPU-инструкции могут влиять на загрузку и результат. Для учебного выпуска достаточно связать model version с `pyproject.toml`, commit SHA и результатом CI; в следующих работах manifest дополнится image digest.

Запретите использовать «у меня загрузилось» как критерий. Приёмочный smoke test должен загрузить модель по registry URI и выполнить прогноз на фиксированном примере.

6. Почему stages больше не используются

MLflow model stages `Staging`, `Production` и `Archived` deprecated с MLflow 2.9 и должны быть заменены aliases и tags. Fixed stages смешивали среду, статус проверки и deployment pointer в одном поле.

В новом подходе:

- alias `candidate` указывает версию, рассматриваемую для выпуска;
- alias `champion` указывает текущую обслуживаемую версию;
- tag `validation_status=passed` фиксирует результат автоматической проверки;
- tag `release_status=approved` фиксирует решение;
- отдельные registered model names могут представлять разные среды при необходимости разграничения доступа.

Не показывайте `transition_model_version_stage()` как рекомендуемый API. Он допустим только в разборе миграции старых материалов.

7. Alias — изменяемый указатель

Model version неизменяема, alias изменяем. URI `models:/flight-delay-classifier@champion` позволяет сервису запрашивать выбранную версию без изменения кода. Это удобно, но создаёт ответственность: переназначение alias является release-операцией и должно оставлять аудит.

Обсудите два режима загрузки:

- разрешить alias только при старте процесса и записать resolved version в лог;
- разрешить периодическое обновление, но тогда нужны атомарная замена, health check и откат.

Alias не заменяет immutable reference в manifest. В manifest сохраняют конкретный номер версии, а alias описывает текущее решение.

8. Tags описывают состояние, но не доказывают его

Tag — метаданные. Любой пользователь с правом записи может поставить `validation_status=passed`. Поэтому tag должен ссылаться на evidence: run ID, отчет проверки, commit и CI run. В учебном проекте значение принимается только вместе с воспроизводимым JSON-отчётом.

Рекомендуемый минимум version tags:

- `validation_status`: `pending`, `passed` или `failed`;
- `source_commit`: полный Git SHA;
- `data_sha256`: hash подготовленных данных;
- `quality_report`: путь или URI отчёта;
- `release_status`: `candidate`, `approved`, `rolled_back`;
- `owner_team`: код команды.

Не помещайте в tags secrets, персональные данные и длинные отчёты.

9. Candidate/champion workflow

Базовая последовательность:

1. Prefect flow завершается успешно и создаёт `run-summary.json`.
2. Модель логируется в MLflow с signature и input example.
3. Создаётся новая model version.
4. Version получает tags происхождения и alias `candidate`.
5. Автоматический gate проверяет метрики, signature, загрузку и smoke prediction.
6. Другой участник выполняет review evidence.
7. Уполномоченный участник переназначает `champion` на candidate.
8. Manifest сохраняет новую и предыдущую champion version.

Важно: candidate может обновляться много раз, champion — только после явного решения.

10. Quality gate — это решение по контракту

Gate должен иметь входы, пороги и объяснимый результат. Недостаточно проверить только ROC AUC. Для ЛР10 минимум включает:

- успешный upstream flow;
- test ROC AUC не ниже утверждённого порога;
- наличие signature и input example;
- загрузку версии из Registry;
- smoke prediction;
- совпадение data hash и commit с evidence;
- отсутствие secrets в manifest.

Порог задаётся в конфигурации или команде запуска и отражается в отчёте. Изменение порога — отдельное reviewable решение, а не ручная правка результата.

11. Автоматизация не отменяет approval

Зелёный gate означает «объявленные проверки пройдены», а не «модель безопасна во всех условиях». Выпуск может учитывать latency, стоимость, grouped metrics, юридические ограничения и доступность rollback. Поэтому отделите машинное доказательство от человеческого решения.

В учебном процессе автор candidate не должен единолично выполнять approval. Reviewer проверяет evidence, а имя или GitHub-логин approving participant фиксируется в manifest. Для команды из двух человек допускается перекрёстное review.

12. Release manifest

Manifest — машинно читаемый снимок решения о выпуске. Минимальная структура:

```
{
  "schema_version": "1.0.0",
  "model_name": "flight-delay-classifier",
  "model_version": "3",
  "aliases": ["candidate", "champion"],
  "previous_champion_version": "2",
  "mlflow_run_id": "...",
  "source_commit": "...",
  "data_sha256": "...",
  "quality_gate": {"passed": true, "minimum_roc_auc": 0.7},
  "decision": {"status": "approved", "approved_by": "student-login"}
}
```

Manifest не должен содержать токен доступа, содержимое dataset или локальный абсолютный путь преподавателя. Пути внутри репозитория допустимы; внешние артефакты указываются проверяемым URI.

13. Promotion как транзакция

Логически promotion состоит из проверки preconditions, получения текущего champion, переназначения alias и записи manifest. Между этими шагами возможен отказ. В production нужен механизм блокировки или optimistic concurrency; в учебной работе достаточно запретить параллельные promotion и проверить, что candidate не изменился между review и командой выпуска.

Перед переназначением сохраните номер предыдущего champion. После переназначения снова прочитайте alias и убедитесь, что он указывает ожидаемую версию. Только затем manifest получает `status=approved`.

14. Rollback без переобучения

Rollback не создаёт новую модель и не повторяет обучение. Он переназначает `champion` на заранее известную проверенную version. Команда должна заранее знать target version и причину отката.

Порядок:

1. зафиксировать инцидент или контролируемый учебный сценарий;
2. прочитать текущий champion;
3. проверить существование target version;
4. переназначить alias;
5. выполнить load и smoke prediction;
6. создать rollback manifest;
7. пометить отозванную версию tag, не удаляя историю.

Удаление плохой версии до разбора разрушает аудит.

15. Контролируемый отказ

Для демонстрации задайте недостижимый порог ROC AUC или удалите обязательное поле из копии summary. Ожидается, что новая version не получит `champion`, текущий champion останется прежним, а failure report объяснит точную причину.

Не имитируйте отказ удалением общей MLflow database. Сценарий должен быть обратимым и не разрушать evidence других участников.

16. Имена и области ответственности

Имя registered model описывает продукт и задачу, например `team-03-flight-delay-classifier`. Не включайте случайный run ID в имя: для этого существуют versions. Для разных команд нужен namespace, чтобы они не переназначали aliases друг друга.

У владельца модели, reviewer и release operator разные задачи. В команде 2–5 человек роли можно совмещать, но author и reviewer конкретного изменения различаются. Роли ротируются относительно ЛР9.

17. Диагностика типичных отказов

Симптом	Вероятная причина	Проверка
Registry недоступен	backend не database-backed или неверный URI	вывести tracking/registry URI
Version создана, модель не грузится	потерян artifact store или environment	открыть source и загрузить exact version
Alias указывает не туда	конкурентный promotion или ручная правка	прочитать version by alias и аудит
Gate passed, manifest failed	сериализация пути или отсутствующее поле	проверить schema manifest
Rollback повторяет проблему	target не прошёл smoke в текущей среде	загрузить exact target до alias update

Учите начинать с exact identifiers: model name, version, run ID и commit. Скриншот страницы Registry не заменяет эти значения.

18. Демонстрационный сценарий

1. Открыть два успешных Prefect run summary.
2. Зарегистрировать их как две версии одного model name.
3. Показать version tags и source run.
4. Назначить лучшей версии alias `candidate`.
5. Выполнить gate и smoke prediction.
6. Назначить `champion`, сохранив previous champion.
7. Загрузить `models:/<name>@champion` и exact version.
8. Выполнить rollback на предыдущую version.
9. Сопоставить два manifest и журнал действий.

19. Вопросы аудитории

1. Почему максимальный номер version не означает лучшую модель?
2. Чем alias отличается от tag?
3. Почему в manifest сохраняется exact version, если сервис использует alias?
4. Что произойдёт, если candidate переназначили после review?
5. Какие проверки нельзя выразить одной метрикой?
6. Почему rollback не должен запускать retraining?
7. Какие данные запрещено помещать в tags и manifest?

20. Связь с ЛР10 и ЛР11

В ЛР10 команда создаёт локальный Registry, регистрирует минимум две версии, применяет `candidate` и `champion`, сохраняет positive и rejected gate reports, формирует release manifest и демонстрирует rollback. Каждый студент защищает собственную version, проверку, review или rollback-сценарий.

В ЛР11 resolved model version станет обязательным label/полем структурированного лога и позволит связать метрики сервиса с конкретным выпуском.

21. Официальные источники

- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/
- MLflow Registry tutorial: https://mlflow.org/docs/latest/ml/model-registry/tutorial
- MLflow Client API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html
- MLflow 3 migration guide: https://mlflow.org/docs/latest/ml/mlflow-3/
- MLflow package releases: https://pypi.org/project/mlflow/